Bee Dash – Concept Document

Tiago Pinto m68106

Pitch Line

Lead your swarm to the hive — the sweetest race in nature is on!

Introduction

Bee Dash is a 2D racing game with single-player and multiplayer support. Each player controls a mother bee and her swarm of daughter bees in a race to be the first to reach the hive. To win, players must collect all 9 pots of honey scattered along the course while avoiding traps that can hinder their progress and affect their final score. The player who completes the race in the fastest time is the winner.

Demographic Breakdown

With its simple control scheme, this racing game is accessible to players of all ages. It is available exclusively on PC.

Feature List

- 2D racing gameplay.
- Simultaneous control of a Queen Bee and her swarm.
- Single-player mode and local competitive multiplayer mode (up to 4 players).
- Main Objective: Collect all 9 honey jars scattered across the track.
- Dynamic obstacles and traps that hinder progression and affect the final score.
- Scoring system based on total time and collection efficiency.
- Colorful and animated visual style, inspired by retro games.
- Final scoreboard showcasing each player's individual performance.

Feature List Breakdown

• 2D Racing Gameplay: Bee Dash is a 2D racing game that combines speed, precision, and control. Players race through a course where every second counts.

• Control a Queen Bee and Her Swarm: The player controls a central queen bee, accompanied by a small swarm of drone bees that follow her automatically.

• Single-Player and Multiplayer Modes: The game features both single-player and local multiplayer modes for up to 4 players. In single-player mode, you can practice, hone your skills on the track, and try to beat your personal best. In local multiplayer, players take turns racing on the same track, and the one who finishes with the fastest time wins.

• Collect 9 Honey Jars: In each race, 9 honey jars are strategically scattered throughout the course. The player must collect all of them before reaching the finish line. The placement of these jars challenges players to explore alternate paths and balance risk and reward.

• Dynamic Traps and Obstacles: The tracks are filled with hazards (such as spinning saws and spikes) that players must avoid. If a player's queen bee touches any of these hazards, their final time will be penalized with a 5-second increase per hit.

• Time and Efficiency-Based Scoring System: The final score is calculated based on the total time taken to complete the race and the player's efficiency in collecting honey.

• Colorful and Animated Visual Style: The game features a vibrant visual design inspired by a retro style, blended with a bee and swarm theme.

• Final Scoreboard: At the end of the game, a final scoreboard is displayed, showing each player's score.

Formal Description

Game State:

The game state contains all relevant information at a specific moment:

- elapsed_time: total race time elapsed (float);
- pots_collected: total number of honey pots collected by the player (int);
- total_pots: total number of honey pots available on the course (int, fixed value = 9);
- player_position: 2D position of the bee on the course (Vector2);
- timer_running: indicates whether the timer is active (boolean);

- current_player: identifies the current player (int).

S = {elapsed_time, pots_collected, player_position, timer_running, current_player}

Game View:

What each player can see:

- Elapsed match time (elapsed_time);
- Pot collection progress (pots collected / total pots);
- The position of their own bee on the course;
- Temporary status messages (e.g., "3/9 Pots Collected").

Players cannot see their opponent's progress during the race, only when it is their own turn to play.

Game Setup:

- elapsed_time = 0
- pots_collected = 0
- player_position = start_position (starting position of the bee)
- timer_running = true

The current player is defined by the "current_player" variable in the Global object.

Player Actions:

Player actions available during the race:

- **Movement:** Change the player's position (player_position) along the course.
- **Honey Pot Collection:** When the bee collides with a pot, the counter is incremented (pots_collected += 1).
- **Time Penalties:** The player receives a penalty (by calling the add_time_penalty(seconds) function) upon colliding with traps.

A = {move(Vector2), collect_pot(), add_time_penalty(seconds)}

Victory Conditions:

A player finishes the race upon collecting all pots (pots_collected >= total_pots) and reaching the hive (the finish line). The winner is determined by the lowest final time recorded via Global.add_result(elapsed_time).

V = victory condition (pots_collected >= total_pots → finish_run)

Progression of Play:

The game state updates as follows:

- Each frame (_process(delta)): If timer_running = true, increment elapsed_time += delta
- When the bee collects a pot (_on_body_entered): Increment pots_collected
- When finishing the run (finish_run):
    1. Set timer_running = false
    2. Save elapsed_time to Global
    3. Advance to next player or show results screen